

Colony-Bounded GNN Hybrid Learning for Cold-Start Fog Service Placement

Rouhollah Nabati^{1*}
Abdulbaghi Ghaderzadeh¹

¹Department of Computer Engineering, Faculty of Engineering,
Islamic Azad University, Sanandaj Branch, Sanandaj, Iran

Abstract

Fog service placement under burst load and membership churn is hard when the control plane itself must form from scratch. Most learning placers either assume an already organized hierarchy or score hosts over a near-global action set, so control cost and cold-start behavior are rarely isolated. We present **DynaCol-GNN**, which keeps DynaCol’s cold-start colony formation, FCM handover, CRT/GRT tables, sticky L1 reuse, and cloud L3 fallback, and changes only L2 host ranking. Candidates are restricted to the pruned CRT (with optional GRT escalation). A two-layer GraphSAGE scorer is mixed with the deterministic DCBO objective through a scale-aware blend that hands control back to exact DCBO when the fog count is large. A flat vector policy is the encoder ablation of the same MDP. In iFogSim2, under normal, burst, and churn traffic, the colony-bounded hybrid cuts mean SLA violations at $N=100$ against DCBO at matched control overhead on both a surveillance DAG and a deeper multipipe DAG. With offline GraphSAGE weights and online fine-tuning (5 trials), DynaCol-GNN also edges the vector ablation on multipipe. A separate 10-trial grid keeps the same direction versus DCBO, though encoder ranking mixes and Holm tests at $n=10$ are not significant. At $N=1000$ the methods stay close with matched overhead. Flat baselines (FogPlan, Greedy-Nearest, Static-DCBO) give absolute SLA context but do not share the cold-start colony control plane.

Keywords: fog computing, service placement, graph neural networks, hybrid learning, self-organizing systems, cold start, CRT/GRT, iFogSim2

1 Introduction

Service placement at the fog and edge must keep end-to-end delay inside SLA limits while demand, membership, and link quality move [Chiang and Zhang \(2016\)](#); [Yousefpour et al. \(2019b\)](#). When load bursts and nodes leave, placers that need a global snapshot, or that assume a ready hierarchy, either spend too much on control or fail early in the run [Ait Salaht et al. \(2020\)](#); [Taleb et al. \(2025\)](#). Reinforcement learning and related adapters help under changing demand [Supraja et al. \(2025\)](#); [Schulman et al. \(2017\)](#), yet many designs still open the action set to almost every host and rarely report the messaging cost of that choice [Hong and Varghese \(2019\)](#).

DynaCol already covers the control-plane side: colonies form from a zero-colony state, an FCM is elected and handed over, and CRT/GRT tables stay bounded. DCBO is the default L2 scorer, with sticky L1 reuse and cloud L3 fallback. Missing is a learner that is *forced* to pick inside those same bounded views.

*Corresponding author: Rouhollah Nabati.

We leave formation, handover, CRT/GRT, L1, and L3 as in DynaCol and replace only L2 ranking. At each decision a GraphSAGE network scores at most the top- K CRT hosts (RTT affinity plus service-DAG context), mixes that score with inverted DCBO utility, and reduces the learning blend to zero once the fabric is large enough that exact DCBO is preferred (Fig. 1). The vector policy is the same MDP without message passing (Fig. 2).

The MDP is colony-bounded ($|A_t| \leq K$ after cold start), so the control plane is part of the learning method. L2 hybrids online scoring with DCBO and hands back to exact DCBO at large N . We compare a HID=16 GraphSAGE encoder (offline import plus fine-tune) with a flat vector ablation on the same features, on iFogSim2 under normal/burst/churn, including a 10-trial journal grid with Holm tests, an $N=1000$ check, and named flat baselines for absolute context.

2 Related Work

2.1 Fog and edge service placement

Surveys of fog placement cover latency, energy, and cost under fixed or centralized controllers [Ait Salaht et al. \(2020\)](#); [Smolka et al. \(2022\)](#); [Taleb et al. \(2025\)](#); [Apat et al. \(2025\)](#); [Chiang and Zhang \(2016\)](#); [Yousefpour et al. \(2019b\)](#). Heuristic and metaheuristic assigners, FogTorch-style QoS matching, and FogPlan Min-Cost assignment improve mapping quality on a given topology [Skarlat et al. \(2017\)](#); [Azimzadeh et al. \(2022\)](#); [Benamer et al. \(2024\)](#); [Brogi and Forti \(2017\)](#); [Yousefpour et al. \(2019a\)](#), but they do not form a self-organizing overlay from an empty colony state.

2.2 Learning-based offloading and placement

DQN/PPO-style and other learning placers adapt online [Mnih et al. \(2015\)](#); [Schulman et al. \(2017\)](#); [Supraja et al. \(2025\)](#), with applications in healthcare/fog stacks, energy-aware modules, and multi-tier containers [Tuli et al. \(2020\)](#); [Hossam et al. \(2024\)](#); [Dogani et al. \(2024\)](#). Resource-management surveys still note large action spaces and the assumption of an organized fabric [Hong and Varghese \(2019\)](#); [Costa et al. \(2022\)](#); [Pallewatta et al. \(2023\)](#). DynaCol-GNN keeps the action set inside CRT/GRT top- K and mixes learning with DCBO so cold start stays stable.

2.3 Self-organizing and hierarchical fog control

Overlay formation and manager election (including DynaCol) separate structure from placement. Hierarchical module managers and evolutionary mappers improve QoS once a hierarchy exists [Mahmud et al. \(2018\)](#); [Guerrero et al. \(2019\)](#), but learning is seldom constrained to the colony view as a hard limit. Here formation stays with DynaCol; learning only re-ranks inside that view.

2.4 Graph learning on infrastructure and service graphs

GraphSAGE and GNN-RL placers encode dependency graphs for scheduling and multi-objective mapping [Hamilton et al. \(2017\)](#); [Lera and Guerrero \(2024\)](#). CRT neighborhoods are small and RTT-structured, so local message passing fits, provided the action set stays colony-bounded. On our multipipe champion lock the GNN beats the vector ablation; we do not claim a large gap on every grid.

2.5 Relation to DynaCol

DynaCol/DCBO is the prior control plane and baseline. We do not restate cold-start formation or FCM handover as new results. The deltas are the colony-bounded L2 MDP, the hybrid GraphSAGE/vector learners with scale-aware blend, and the multipipe / journal / baseline study.

Table 1: Where DynaCol-GNN sits relative to common placement families.

Method family	Cold-start	Bounded view	Hybrid w/ determ. obj.	Local graph/DAG
Classical / GA / ILP	Rarely	No	Sometimes	Rarely
FogPlan / Edgeward / Static-DCBO	No/partial	No/fixed	No	No
DRL placers (DQN/PPO)	Rarely	Rarely	Rarely	Sometimes
GNN-RL placers	Rarely	Rarely	Rarely	Yes
DynaCol / DCBO	Yes	Yes	N/A	No
DynaCol-GNN	Yes	Yes	Yes	Yes

We use iFogSim / iFogSim2 for simulation [Gupta et al. \(2017\)](#); [Mahmud et al. \(2022\)](#) (EdgeCloudSim is a related alternative [Sonmez et al. \(2018\)](#)) and Holm’s procedure for multiple comparisons [Holm \(1979\)](#).

3 Background: DynaCol Control Plane

Only the reused pieces are summarized here.

Nodes join by query/advertise until colonies appear. An FCM is elected by potency and updated with ϵ -greedy probes and a cooldown on handover. Each FCM keeps a CRT of member capacity, availability, RTT, and attractiveness; GRT summaries allow limited cross-colony search without a full host list. Placement is three-tier: sticky L1 reuse if the current host still meets SLA thresholds; DCBO L2 scoring over CRT then GRT; cloud L3 if both fail. DynaCol-GNN replaces only the L2 ranking inside CRT/GRT top- K (Fig. 1).

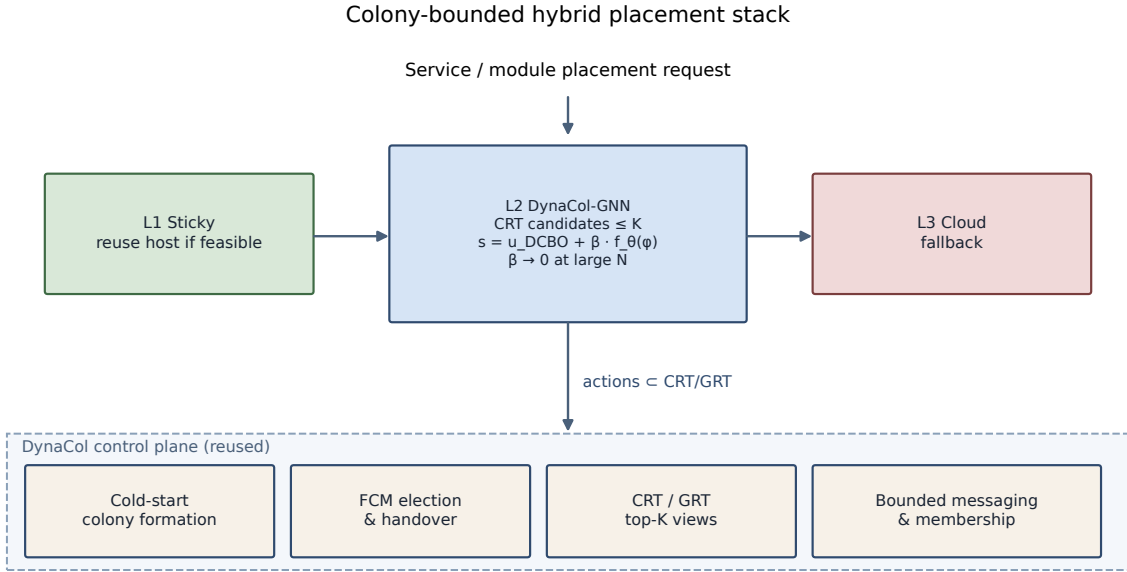


Figure 1: Placement stack on the reused DynaCol control plane: L1 sticky reuse, L2 DynaCol-GNN over CRT/GRT, L3 cloud fallback.

4 Colony-Bounded Hybrid Learning

4.1 MDP

On each FCM reconcile, state S_t is the local CRT (size $\leq K$), optional GRT summaries, and service/DAG features. Action A_t picks a feasible CRT host or escalates to GRT/L3. Reward $R_t \in [0, 1]$ is graph-SLA

shaped for GNN and a weaker TD signal for Vector. The hard limit $|A_t| \leq K$ is intentional.

4.2 Hybrid scoring

$$s(i) = u_{\text{DCBO}}(i) + \beta \cdot f_{\theta}(\phi(i)), \quad (1)$$

with u_{DCBO} the inverted DCBO objective. We take $\arg \max_i s(i)$ and allow a small ε -greedy rate while n_f is below the large-scale gate. The blend β falls with fog count and is set to zero for $n_f \geq 450$, so L2 becomes exact DCBO.

4.3 Encoders

The vector policy (`colony_rl`) concatenates host channels with service/DAG scalars and scores with a linear head. The GNN policy (`colony_gnn`) builds an RTT-affinity graph on CRT candidates,

$$h' = \tanh(W_{\text{self}}h + W_{\text{neigh}}Ah), \quad (2)$$

then reads out from $\text{concat}(h, \text{context})$. Offline JSON weights can be loaded and fine-tuned online. The vector ablation zeros the co-location channel so structure is the main difference (Fig. 2).

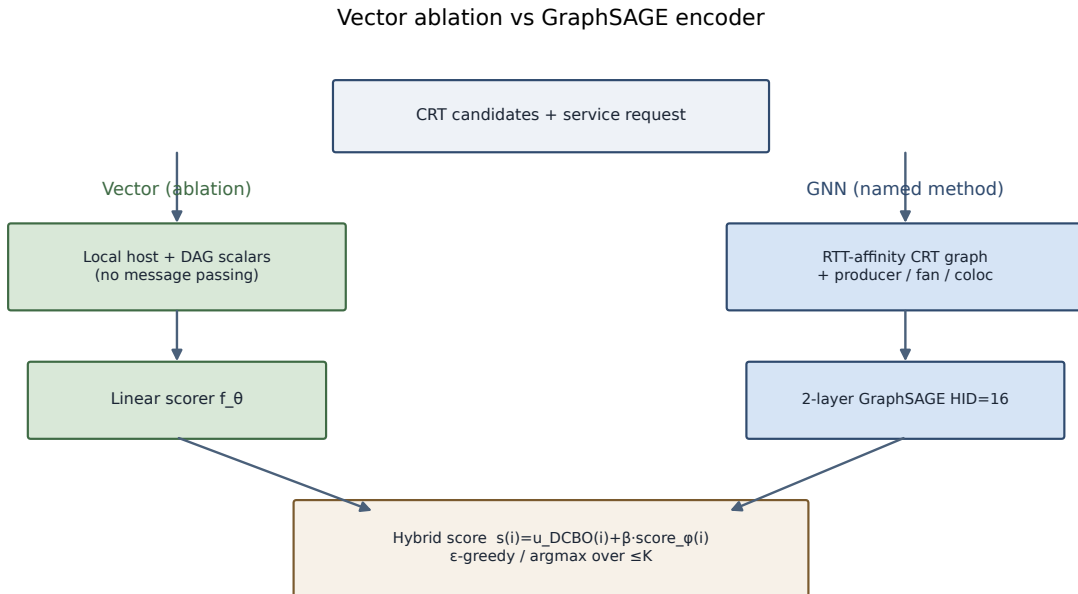


Figure 2: Vector ablation and GraphSAGE pipelines into the same hybrid score (1).

4.4 Updates and offline import

After i^* is chosen, parameters move toward the observed reward unless offline weights are frozen. Attractiveness updates follow DCBO. Logged CRT trajectories train the offline importer outside the simulator and export tensors that match the Java layout.

4.5 Cost and algorithm

Scoring touches at most K candidates at fixed width; messaging is still the CRT/GRT traffic of DynaCol.

Algorithm 1 Colony-bounded hybrid reconcile (sketch)

```
1: procedure RECONCILE( $r$ , fcm, current)
2:   if  $\beta \approx 0$  then
3:     return DCBO.reconcile(...)
4:   end if
5:   if L1 sticky OK then
6:     return current
7:   end if
8:   if CRT enabled then
9:      $C \leftarrow$  top- $K$  feasible CRT entries
10:    pick  $i^*$  by hybrid score on  $\phi(C)$ ; update  $\theta$ 
11:    return host( $i^*$ )
12:  end if
13:  if GRT enabled then
14:    escalate via ranked GRT summaries
15:  end if
16:  return cloud
17: end procedure
```

▷ L3

5 Experimental Setup

Simulations run in iFogSim2 on the shared DynaCol runtime [Gupta et al. \(2017\)](#); [Mahmud et al. \(2022\)](#). We compare `dcbo`, `colony_rl` (Vector), and `colony_gnn` (GNN), plus `static_dcbo`, `fogplan`, and `greedy`. Edgeward was dropped on multipipe after single trials exceeded 25 minutes.

Target sizes are $N \in \{100, 300, 500, 1000\}$ under normal, burst, and churn. The surveillance DAG is the shallow camera pipeline; the multipipe DAG adds fan-out, skip, and join edges (Fig. 3) with SLA deadline scale 10. We report SLA violation %, P95 loop delay, normalized control overhead, and colony count. The champion multipipe lock uses 5 trials; journal grids use 10 trials with paired t and Holm adjustment inside each N [Holm \(1979\)](#); baselines use 5 trials. Champion settings: $HID=16$, $\beta=0.60$, $\varepsilon=0.02$, offline weights with fine-tune.

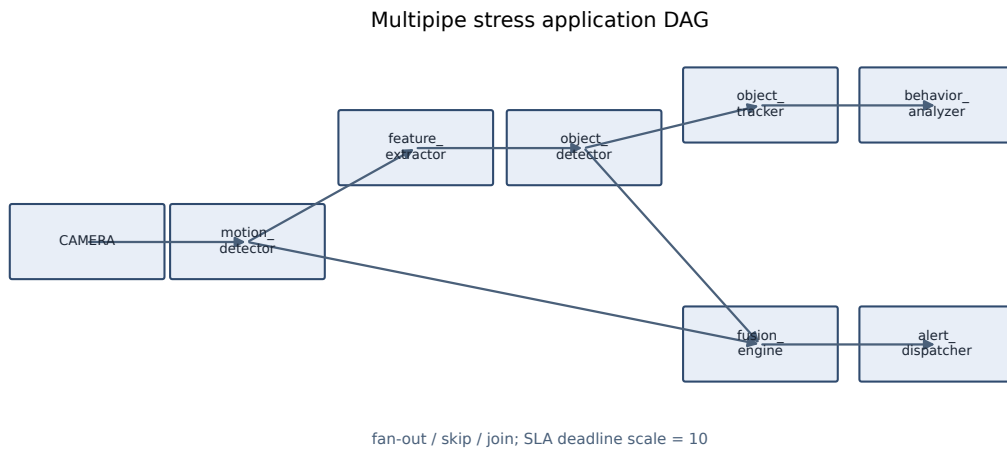


Figure 3: Multipipe application DAG used in the champion and journal grids.

6 Results

6.1 Surveillance DAG

Table 2 and Fig. 4 summarize the surveillance grid. At $N=100$ the hybrid (Vector and GNN coincide on this shallow app) lowers mean SLA against DCBO at matched overhead. At $N \in \{300, 500\}$ the curves meet because the blend gate hands ranking back to DCBO (see also Fig. 7).

Table 2: Surveillance DAG: mean SLA violation % (5 trials).

N	Scenario	Hybrid	DCBO
100	Normal	11.51	13.31
100	Burst	14.02	14.78
100	Churn	14.58	15.37
300	all	$\approx 0.48-0.51$	$\approx 0.48-0.50$
500	all	$\approx 0.57-0.59$	$\approx 0.57-0.59$

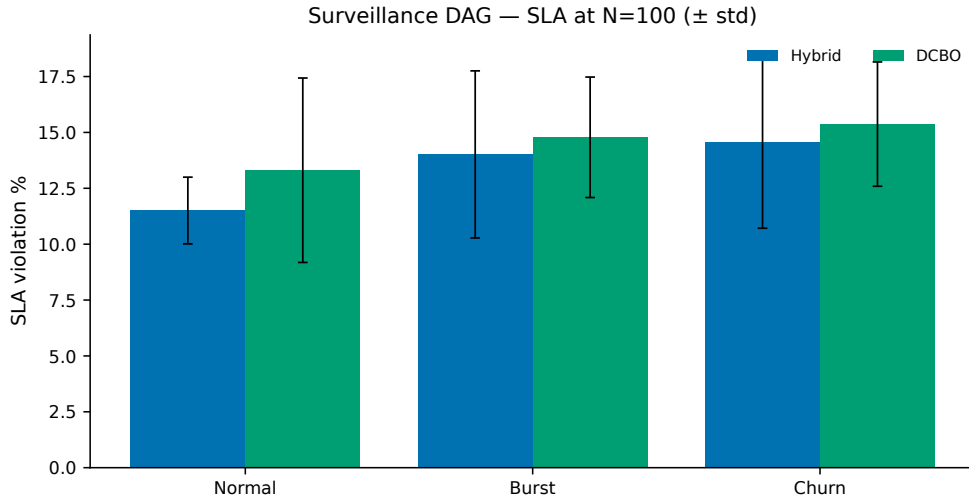


Figure 4: Surveillance SLA at $N=100$ (mean $\pm$ std, 5 trials).

6.2 Multipipe champion lock

Table 3 and Fig. 5 are the 5-trial multipipe lock with imported GraphSAGE weights and fine-tuning. GNN is best on all three loads against Vector and DCBO.

Table 3: Multipipe champion lock: mean SLA % (5 trials, $N=100$).

N	Scenario	GNN	Vector	DCBO
100	Normal	10.64	10.98	11.48
100	Burst	24.89	25.35	26.13
100	Churn	22.31	22.43	23.21

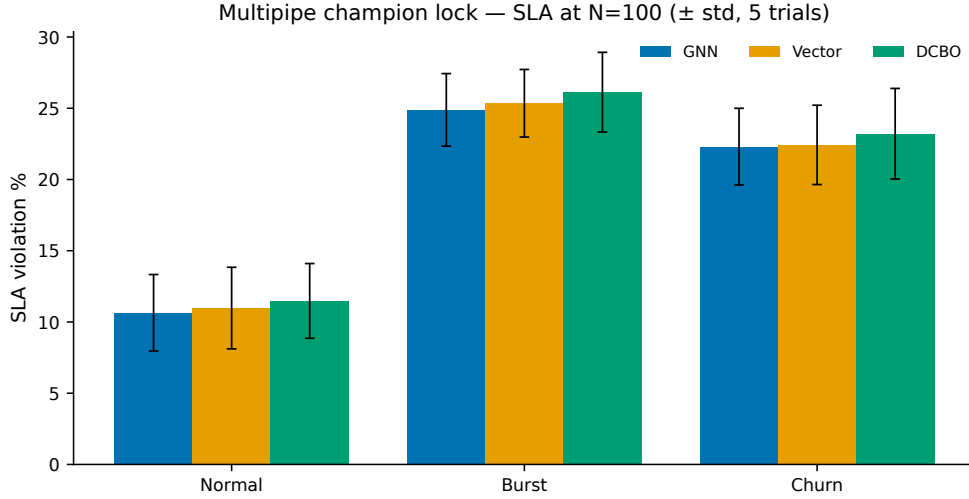


Figure 5: Multipipe champion lock SLA at $N=100$ (5 trials).

6.3 Encoder ablation context

Without imported weights, an earlier online-only multipipe medium put Vector slightly ahead of GNN while both beat DCBO (Table 4). The champion gap in Table 3 appears after structure-weighted offline training plus fine-tune.

Table 4: Earlier online-only multipipe medium (context; pre-champion).

N	Scenario	Online-GNN	Vector	DCBO
100	Normal	10.34	10.25	11.21
100	Burst	24.39	24.09	26.42
100	Churn	21.15	21.13	23.04

6.4 Journal 10-trial grid

The 10-trial multipipe grid (Table 5, Fig. 6) still beats DCBO in mean SLA on every load, but Vector leads on burst and churn. Absolute gaps are about 0.3–1.1 points; Holm tests appear with Fig. 11.

Table 5: Multipipe journal grid: mean SLA % (10 trials, $N=100$).

N	Scenario	GNN	Vector	DCBO
100	Normal	11.09	11.34	11.60
100	Burst	25.89	25.62	26.71
100	Churn	22.91	22.61	23.23

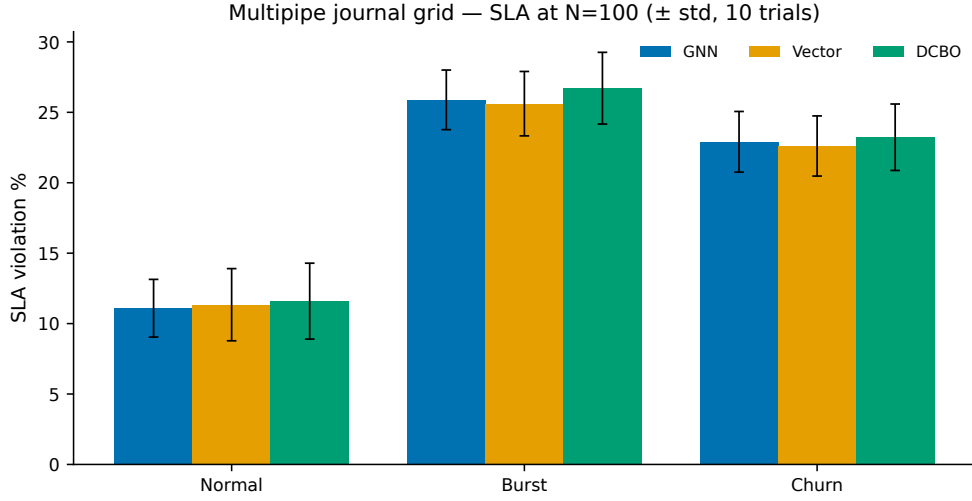


Figure 6: Multipipe journal-grid SLA at $N=100$ (10 trials).

6.5 Scale and overhead

Fig. 7 shows surveillance mean SLA versus target size: the $N=100$ gap closes once the blend gate returns L2 to DCBO. Fig. 8 shows normalized control overhead on the multipipe champion lock; GNN, Vector, and DCBO stay matched.

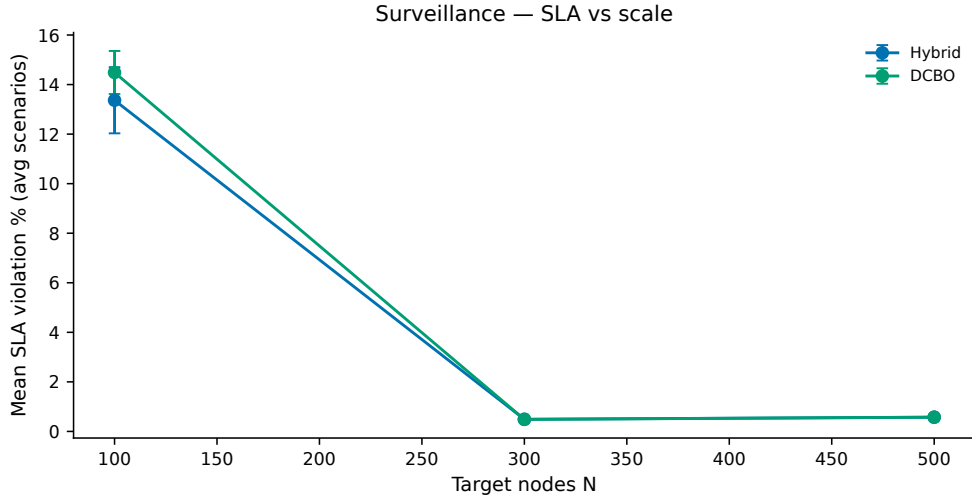


Figure 7: Surveillance mean SLA versus target size N (averaged over scenarios).

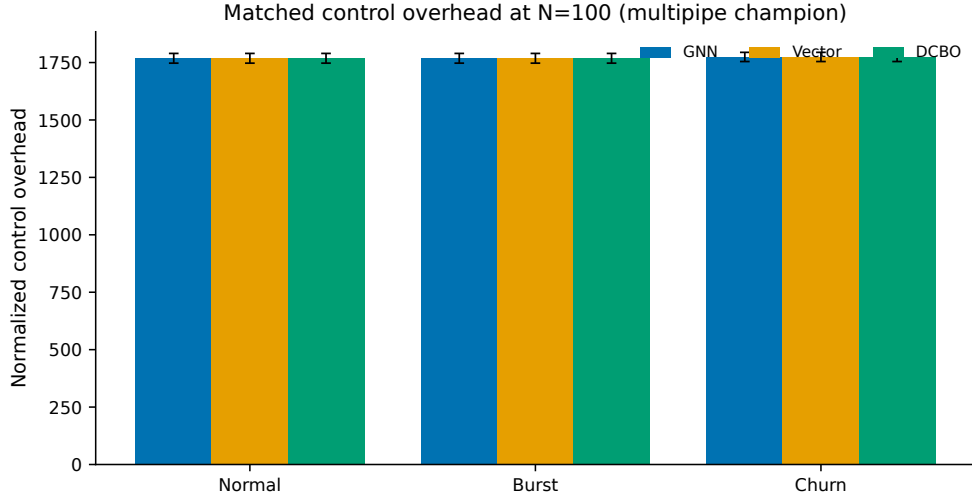


Figure 8: Normalized control overhead for the multipipe champion lock ($N=100$). GNN, Vector, and DCBO remain matched.

6.6 Large- N check

At $N=1000$ (Table 6, Fig. 9) absolute SLA is low and overhead matches (≈ 2112 – 2140). Holm tests are again non-significant after correction. Burst and churn still favor GNN in the means.

Table 6: Multipipe at $N=1000$: mean SLA % (10 trials).

Scenario	GNN	Vector	DCBO	Overhead
Normal	0.82	0.80	0.81	2112
Burst	2.53	3.13	3.59	2112
Churn	1.94	2.27	2.70	2140

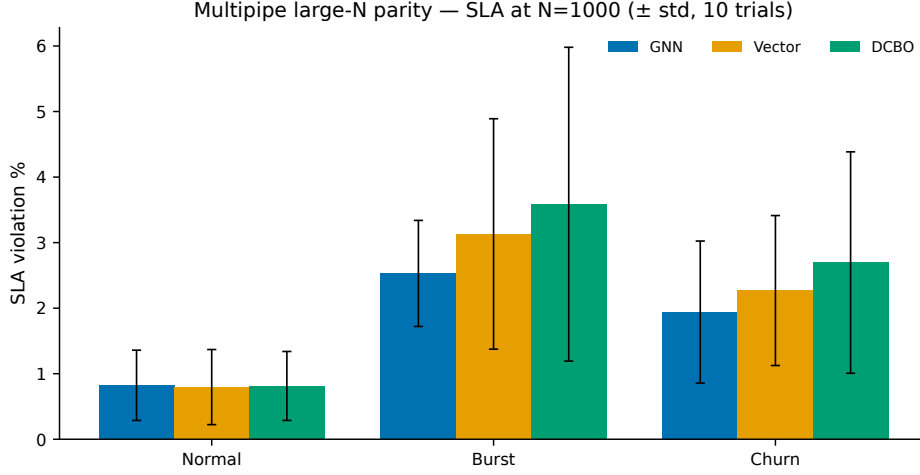


Figure 9: Multipipe SLA at $N=1000$ (mean $\pm$ std, 10 trials).

6.7 Named baselines

FogPlan, Greedy-Nearest, and Static-DCBO can post lower multipipe SLA than the colony policies (Table 7, Fig. 10), but their overhead and colony counts are not comparable: FogPlan/Greedy run with one logical colony and far fewer control messages. The claim we defend is therefore inside the DynaCol plane—GNN vs Vector vs DCBO at matched overhead.

Table 7: Multipipe $N=100$: flat baselines (5 trials) beside the champion colony methods. Overhead is not matched across planes.

Scenario	FogPlan	Greedy	Static	GNN	Vector	DCBO
Normal	8.28	7.32	9.25	10.64	10.98	11.48
Burst	23.24	22.63	21.61	24.89	25.35	26.13
Churn	19.88	19.37	19.02	22.31	22.43	23.21
Overhead	253	659	760	≈ 1769	≈ 1769	≈ 1769
Colonies	1	1	19	≈ 7.6	≈ 7.6	≈ 7.6

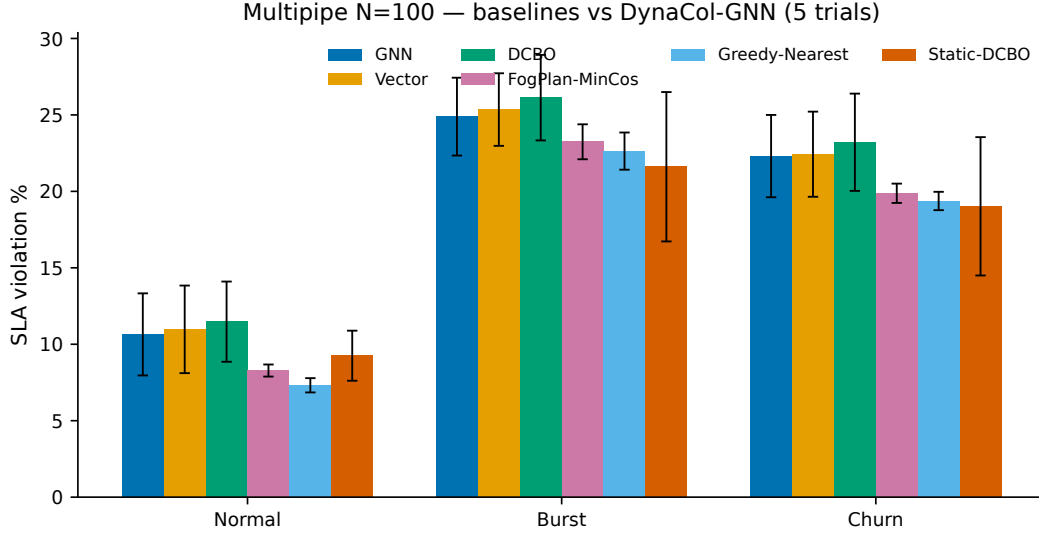


Figure 10: Flat baselines versus colony-bounded methods on multipipe $N=100$ (5 trials).

6.8 Holm tests

Holm-adjusted paired tests inside the $N=100$ journal family (Fig. 11) do not reach $\alpha=0.05$.

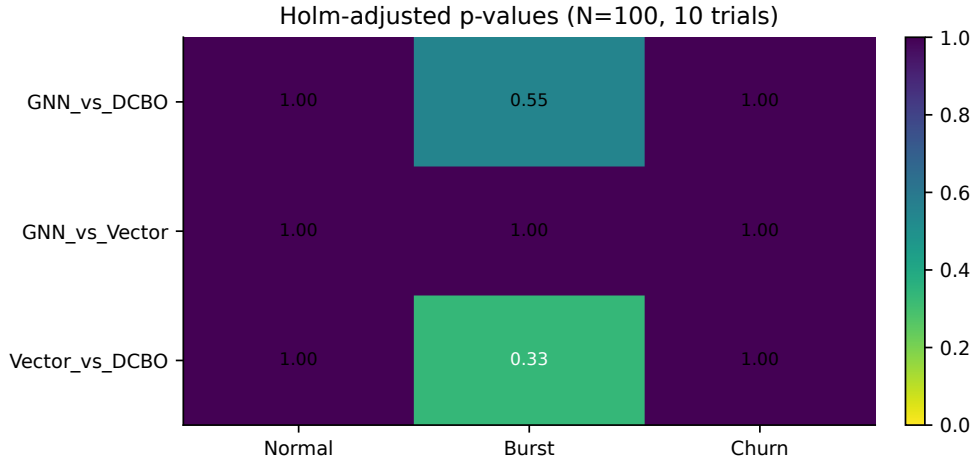


Figure 11: Holm-adjusted p -values for paired SLA tests at $N=100$ (10 trials).

6.9 Scale-aware blend

The gate uses the live fog count. Threshold 450 avoids treating a target $N=500$ topology that yields about 497 devices as “still learning.” When $\beta \approx 0$, L2 is DCBO by construction.

7 Discussion

Bounding $|A_t|$ ties learning to the same CRT/GRT traffic DynaCol already exchanges. Inside that plane, overhead matches DCBO while $N=100$ SLA improves (Tables 2–3).

The title puts GNN first because the named method is GraphSAGE hybrid ranking. The 5-trial champion lock supports a GNN edge over Vector; the 10-trial means are softer and Holm tests at $n=10$ do not clear $\alpha=0.05$. That is a power issue as much as a method issue, and we report it as such.

Flat baselines in Table 7 are useful for absolute SLA, not for a same-plane ranking. We also do not claim new cold-start formation, new FCM election, or testbed validation. Limits include simulation-only evidence, calibrated multipipe deadlines, logged (not fully on-policy) offline data, and the omission of Edgeward on multipipe for runtime reasons.

8 Conclusion

DynaCol-GNN keeps fog placement inside cold-start colony views: L2 mixes GraphSAGE scoring with DCBO, actions stay in CRT/GRT top- K , and large fabrics fall back to exact DCBO. On iFogSim2 the hybrid lowers SLA at $N=100$ versus DCBO at matched overhead on surveillance and multipipe workloads, and stays close at $N=1000$. The champion multipipe lock also favors GNN over the vector ablation; the 10-trial journal grid remains directionally better than DCBO without Holm significance at $n=10$. Flat baselines remind the reader that absolute SLA and control-plane cost can trade off outside the colony plane.

Next steps include CRT-ablated learning, stronger on-policy offline training, same-plane baselines with matched messaging, and testbed runs.

A Hyperparameters

Table 8: Locked-grid hyperparameters.

Parameter	Default / note
CRT top- K	DynaCol TOP_K_CRT_CANDIDATES
ε	0.02 on champion multipipe (0 at large N)
Vector / GNN LR	0.05 / 0.04
GNN hidden	16, 2 layers, tanh
Blend (multipipe)	0.60
Multipipe SLA scale	10.0
Large- N gate	$n_f \geq 450 \rightarrow$ exact DCBO
Trials	5 (champion/baselines); 10 (journal)

References

- Aït Salaht, F., Desprez, F., and Lebre, A. (2020). An overview of service placement problem in fog and edge computing. *ACM Computing Surveys*, 53(3):65:1–65:35.
- Apat, H. K., Goswami, V., Sahoo, B., Barik, R. K., and Saikia, M. J. (2025). Fog service placement optimization: A survey of state-of-the-art strategies and techniques. *Computers*, 14(3):99.
- Azimzadeh, M., Rezaee, A., Jassbi, S. J., and Esnaashari, M. (2022). Placement of IoT services in fog environment based on complex network features: A genetic-based approach. *Cluster Computing*, 25(5):3423–3445.
- Benamer, A. R., Hadj-Alouane, N. B., Boussetta, K., and Hadj-Alouane, A. B. (2024). A genetic algorithm-based approach for placement in the fog of latency-sensitive multiplayer game servers. *Cluster Computing*, 27:11249–11275.
- Brogi, A. and Forti, S. (2017). Qos-aware deployment of IoT applications through the fog. *IEEE Internet of Things Journal*, 4(5):1185–1192.

- Chiang, M. and Zhang, T. (2016). Fog and iot: An overview of research opportunities. *IEEE Internet of Things Journal*, 3(6):854–864.
- Costa, B., Bachiega Jr., J., de Carvalho, L. R., and Araujo, A. P. F. (2022). Orchestration in fog computing: A comprehensive survey. *ACM Computing Surveys*, 55(2).
- Dogani, J., Yazdanpanah, A., Zare, A., and Khunjush, F. (2024). A two-tier multi-objective service placement in container-based fog-cloud computing platforms. *Cluster Computing*, 27(4):4491–4514.
- Guerrero, C., Lera, I., and Juiz, C. (2019). Evaluation and efficiency comparison of evolutionary algorithms for service placement optimization in fog architectures. *Future Generation Computer Systems*, 97:131–144.
- Gupta, H., Vahid Dastjerdi, A., Ghosh, S. K., and Buyya, R. (2017). ifogsim: A toolkit for modeling and simulation of resource management techniques in the internet of things, edge and fog computing environments. In *Proc. of IEEE/ACM International Conference on Utility and Cloud Computing (UCC)*, pages 47–56.
- Hamilton, W. L., Ying, R., and Leskovec, J. (2017). Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70.
- Hong, C.-H. and Varghese, B. (2019). Resource management in fog/edge computing: A survey on architectures, infrastructure, and algorithms. *ACM Computing Surveys*, 52(5).
- Hossam, H. S., Abdel-Galil, H., and Belal, M. (2024). An energy-aware module placement strategy in fog-based healthcare monitoring systems. *Cluster Computing*, 27:7351–7372.
- Lera, I. and Guerrero, C. (2024). Multi-objective application placement in fog computing using graph neural network-based reinforcement learning. *The Journal of Supercomputing*, 80:27073–27094.
- Mahmud, R., Pallewatta, S., Goudarzi, M., and Buyya, R. (2022). ifogsim2: An extended ifogsim simulator for mobility, clustering, and microservice management in edge and fog computing environments. *Journal of Systems and Software*, 190:111306.
- Mahmud, R., Ramamohanarao, K., and Buyya, R. (2018). Latency-aware application module management for fog computing environments. *ACM Transactions on Internet Technology*, 19(1):1–21.
- Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518:529–533.
- Pallewatta, S., Kostakos, V., and Buyya, R. (2023). Placement of microservices-based iot applications in fog computing: A taxonomy and future directions. *ACM Computing Surveys*, 55(14s):321:1–321:43.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. In *arXiv preprint arXiv:1707.06347*.
- Skarlat, O., Nardelli, M., Schulte, S., and Dustdar, S. (2017). Towards qos-aware fog service placement. In *Proc. IEEE 1st International Conference on Fog and Edge Computing (ICFEC)*, pages 89–96.
- Smolka, S. et al. (2022). Evaluation of fog application placement algorithms: A survey. *Computing*, 104:2373–2415.

- Sonmez, C., Ozgovde, A., and Ersoy, C. (2018). Edgecloudsim: An environment for performance evaluation of edge computing systems. *Transactions on Emerging Telecommunications Technologies*, 29(11):e3493.
- Supraja, T., Chawla, P., and Gill, S. S. (2025). AI-Driven Service Placement in Fog and Edge Computing Environments: A Systematic Literature Review, Taxonomy and Future Directions. *Cluster Computing*, 28(16):1070.
- Taleb, I., Guillaume, J.-L., and Duthil, B. (2025). A survey on services placement algorithms in integrated cloud-fog/edge computing. *ACM Computing Surveys*, 57(11).
- Tuli, S., Basumatary, N., Gill, S. S., Kahani, M., Arya, R. C., Wander, G. S., and Buyya, R. (2020). HealthFog: An ensemble deep learning based smart healthcare system for automatic diagnosis of heart diseases in integrated IoT and fog computing environments. *Future Generation Computer Systems*, 104:187–200.
- Yousefpour, A., Fung, C., Nguyen, T., Kadiyala, K., Jalali, F., et al. (2019a). Fogplan: A lightweight qos-aware dynamic fog service provisioning framework. *IEEE Internet of Things Journal*, 6(3):5080–5096.
- Yousefpour, A., Fung, C., Nguyen, T., Kadiyala, K., Jalali, F., Niakanlahiji, A., Kong, J., and Jue, J. P. (2019b). All one needs to know about fog computing and related edge computing paradigms: A complete survey. *Journal of Systems Architecture*, 98:289–330.